

FormaTexto — Launch & Deployment Guide

July 2026

The core architectural constraint

Your backend **cannot** go on serverless (Vercel Functions, AWS Lambda, Cloudflare Workers). Three reasons, all real, all in your own code:

1. **LibreOffice shell-out** (`docxToPdf.ts`, `paginateSumario.ts`) — needs a persistent OS binary, not a stateless function.
2. **In-process `JobQueue`** (`jobQueue.ts`, max 2 concurrent) — lives in memory across requests. A serverless cold-start per request would break the concurrency limit entirely.
3. **Long-running AI pipeline** — a full thesis is 30–40+ AI calls; can take ~30 minutes (that's what set the 2-hour stuck-job threshold). Most serverless platforms cap execution at 10–60 seconds.

So: **frontend = static/serverless is fine. Backend = one persistent server, always.**

Frontend deployment

`web/` is a standard Vite build (`npm run build` → static `dist/`). Deploy to **Vercel** or **Netlify** (free tier covers you well past your first hundreds of users).

- Set env vars in the host's dashboard: `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`, `VITE_STRIPE_PUBLISHABLE_KEY`, `VITE_API_URL` (your backend's public URL).
- Because it's a React Router SPA, you need a rewrite rule so all paths serve `index.html` (Vercel/Netlify both auto-detect this for Vite, but confirm `/projects/abc123` doesn't 404 on refresh).
- Custom domain: point it here, not at the backend.

Backend deployment

You need a host that runs a **long-lived Docker container**, not a serverless function. Best fits for your size: **Railway**, **Render**, or **Fly.io** — all support "always-on" containers, cost roughly US\$7–25/mo at low traffic, and let you install system packages via Dockerfile (unlike most PaaS buildpacks).

You don't have a `Dockerfile` yet — that's the one missing artifact standing between you and deploying. It needs to be:

```
FROM node:20-slim
RUN apt-get update && apt-get install -y libreoffice-writer fonts-liberation fonts-dejavu &&
  rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
```

```
COPY . .
RUN npm run build
ENV SOFFICE_PATH=/usr/bin/soffice
CMD ["npm", "start"]
```

That single `apt-get install libreoffice-writer` line is what closes the open item "PDF export / LibreOffice has no production home" — it may need **zero application code**, just this Dockerfile.

Environment variables — what changes for production

Everything in `server/.env.example`, with these production-specific values:

Var	Production value
<code>STRIPE_SECRET_KEY</code>	live key (<code>sk_live_...</code>), not test
<code>STRIPE_WEBHOOK_SECRET</code>	new one — from a webhook endpoint you create in the Stripe Dashboard pointing at <code>https://your-backend/api/webhook</code> , not the local <code>stripe listen</code> secret
<code>TRUST_PROXY</code>	<code>1</code> for Railway/Render/Fly (single reverse-proxy hop) — currently unset; leaving it unset in prod silently breaks per-IP rate limiting
<code>FRONTEND_URL</code>	your real domain, for CORS
<code>SOFFICE_PATH</code>	<code>/usr/bin/soffice</code> (matches the Dockerfile above)
<code>OPENROUTER_API_KEY</code>	same key, but add credits — free tier is 50 req/day account-wide, a single thesis burns most of that
<code>SUPABASE_SERVICE_ROLE_KEY</code>	production Supabase project's key — never put this in the frontend build
<code>WEBHOOK_SECRET</code>	a fresh random string, used to gate <code>/api/maintenance/cleanup-expired</code> and <code>/api/notifications/project-ready</code>
<code>RESEND_API_KEY</code>	same key; but see "email" below

Secrets go in the host's env var dashboard (Railway/Render both encrypt these at rest), never committed.

Supabase — going from dev to production

- Create a **separate production Supabase project** (don't reuse your dev one — dev data, dev users, and dev RLS testing shouldn't touch real payments).
- Run the RLS policies (already verified secure by your own audit — `projects/orders/user_profiles` all correctly scoped).

- Run the two pending migrations: `processing_attempts` and `processing_started_at` columns.
- Run `server/sql/retry_pending_cron.sql` — but **leave the cron OFF** at first launch (it burns fresh OpenRouter quota retrying old jobs); turn it on once you're past the free tier.
- Run `server/sql/cleanup_cron.sql` for the 30-day file auto-deletion, plus its two Vault secrets.
- Set a Storage bucket file-size limit on `projects` (dashboard setting) — an oversized upload can hang LibreOffice conversion.

Stripe — production

- Switch to live mode, live publishable/secret keys.
- Create the production webhook endpoint in the Stripe Dashboard (Developers → Webhooks), subscribe it to the payment-intent-succeeded event, copy its secret into `STRIPE_WEBHOOK_SECRET`.
- Test with a real small card charge before announcing — webhook plumbing is the single most common "works in dev, silently fails in prod" spot (order recording + trial consumption both depend on it).

Email

Still sending from `onboarding@resend.dev` — reads as untrustworthy for a paid service. Before launch: verify a real domain in Resend (a few DNS TXT/CNAME records), swap the `from` address. Cheap, ~15 minutes, meaningfully raises trust for a payment product.

Security — what's already done vs. what's left

Already shipped: payment-bypass gate, checkout auth via verified Bearer token, baseline security headers, `x-powered-by` disabled, 1MB body cap, rate limiting on processing/checkout/sensitive routes, SSRF-hardened `/documents/fetch`, RLS-scoped storage. That's a real security pass — good foundation.

Left before going live:

- Merge `security-testing` → `main` (not merged yet).
- Set `TRUST_PROXY` correctly for whichever host you pick, or the rate limiter degrades to one shared budget.
- Operator alerting (Sentry or similar) — currently a failed job just `console.errors` into the void. Cheap insurance for the first weeks.
- A refund/redo path — no mechanism today for "this output is wrong, fix it." High-stakes product (a student's thesis), worth having before real money moves.

Cost estimate at low volume (first ~50–200 customers/month)

Item	Cost
Frontend (Vercel/Netlify free tier)	R\$0

Item	Cost
Backend container (Railway/Render, always-on) ~US\$7–25/mo (R\$40–140)	
Supabase (free tier covers early volume; Pro if storage/bandwidth grows)	R\$0–125/mo
OpenRouter (paid tier, 1000 req/day)	usage-based, likely R\$50–300/mo depending on volume — validate real per-doc AI cost first
Resend (free tier: 3,000 emails/mo)	R\$0
Stripe	3.99% + R\$0.39 per transaction (no fixed cost)
Domain	~R\$40/year

Realistic fixed floor: ~R\$100–300/month before per-document AI variable cost. Don't set final pricing until you've run a real ~40-page validation document.

Launch order (do these top to bottom)

1. Merge `security-testing` → `main`.
2. Write + test the Dockerfile (LibreOffice in the image).
3. Stand up production Supabase project, run migrations + RLS + storage size limit.
4. Deploy backend (Railway/Render/Fly), set all prod env vars, confirm `TRUST_PROXY`.
5. Deploy frontend, point `VITE_API_URL` at the live backend.
6. Stripe live keys + production webhook, test with a real small charge.
7. Verify Resend domain, swap `from` address.
8. Add credits to OpenRouter.
9. Process one full real ~40-page document end-to-end in production before opening signups — confirms LibreOffice-in-Docker, page numbering, sumário, PDF export, webhook, and email all actually work together outside your dev machine.
10. Add Sentry (or equivalent) before, not after, real traffic.

SEO — starting from zero

Being blunt about where things stand: `web/index.html` currently has a generic `<title>forma-texto</title>`, no meta description, no Open Graph tags, no sitemap, no robots.txt, and no blog/content route exists in `routes.ts` at all. "Optimize for SEO" today actually means two separate jobs: fix the technical basics on the app you have, and build the content engine that doesn't exist yet.

1. The technical problem specific to your stack: it's a client-rendered SPA

Google *can* crawl React SPAs now, but with real costs: slower indexing (pages queue for a secondary

rendering pass), and **social/messaging link previews often fail** — WhatsApp/Facebook/Twitter unfurl by reading `<meta>` tags in the raw HTML, without running JavaScript, so every link shared in a student WhatsApp group would show a blank preview card. That directly undercuts the referral channel the marketing plan leans on.

Two fixes, differently scoped:

- **Cheap fix for the app pages** (`/`, `/get-started`, etc.): add `react-helmet-async` so each route sets its own `<title>`/`meta` description dynamically, and hand-write good static OG tags in `index.html` for the landing page (the one most likely to be shared raw). Doesn't fix crawl-speed for logged-in/dynamic pages, but those don't need to rank anyway.
- **Real fix for content pages** (the blog you don't have yet): don't build it inside the React SPA. Build it as a separate statically-generated site (Astro, or plain markdown → static HTML) served at `/blog` or a `blog.` subdomain. Static HTML is what actually wins ABNT-guide search rankings — this is also cheaper and simpler than adding SSR to the whole app.

2. What to actually build (grounded in the proven channel)

Mettzer's whole acquisition engine is evergreen guides for exact-match queries. To compete you need, in order:

1. **A `/blog` (or subdomain) static site** — separate from the React app, so it's fast and fully crawlable without JS.
2. **1–2 posts/week** targeting the query space directly: "normas ABNT 2026", "como formatar TCC", "sumário ABNT", "citação longa ABNT", "margens e espaçamento ABNT", "capa ABNT modelo". Each post: a real how-to, a before/after image of a page processed by your pipeline, and a CTA to the 1-page free trial.
3. **On-page basics per post**: descriptive `<title>` (Portuguese, includes the exact query phrase), meta description, one `<h1>`, semantic `<h2>`s for subsections, `Article` structured data (JSON-LD) so Google can show rich snippets.
4. **sitemap.xml + robots.txt** in `web/public/` — neither exists today. Sitemap should list the app's public routes (landing, terms, privacy) plus every blog post; submit it in Google Search Console once it exists.
5. **Core Web Vitals** — since the marketing site should be static, this should already be fast; just don't ship a huge unoptimized hero image (Vite's asset pipeline mostly handles this for free on the app side).
6. **hreflang tags** across your three locales (`pt-BR`, `pt-PT`, `en`) — i18next infrastructure already exists; SEO just needs `<link rel="alternate" hreflang="pt-BR" ...>` tags added so Google doesn't treat the three as duplicate content.
7. **Backlinks, the realistic way for this niche**: get listed on university library "TCC resources" pages, get mentioned by education micro-influencers, and answer real ABNT questions on Brazilian Q&A sites (Quora-BR, Reddit) with a link back where genuinely relevant — not spam, actual answers.

Suggested next step

This is worth tackling in two separate passes rather than one big change: **(a)** the quick technical fix — meta tags, sitemap, robots.txt on the existing app, roughly an hour of work — and **(b)** standing up the actual blog, a bigger decision (static site generator choice, hosting, content pipeline) worth scoping properly before writing code. Both are planned for a dedicated branch later.